1. **The data structure used to implement recursive function calls**: d) stack
2. **Iteration requires more system memory than recursion.** b) False
3. **Recursion is a method in which the solution of a problem depends on**: c) Smaller instances of the same problem
4. **Recursion is similar to which of the following?** b) Loop
5. **Define recursion. When recursion can be used.**

   Recursion is a programming technique in which a function calls **itself** either **directly** or **indirectly** to solve a problem. Each recursive call solves a **smaller sub-problem** of the original, and the process continues until a **base case** is reached that can be solved without further recursion.

   When Recursion Can Be Used:
   Recursion is useful and appropriate when:
   1. **The problem can be broken down into smaller sub-problems of the same type** (e.g., factorial, Fibonacci, tree traversal).
   2. **The size of the input decreases with each recursive call**, eventually reaching a base case.
   3. **The solution to the problem can be expressed in terms of the solution to smaller instances** (recursive relation).
   4. **When using data structures like trees or graphs**, where recursion simplifies traversal.
   5. **When iterative solutions are more complex or less intuitive**, recursion may offer clarity.
6. **Recall the limitations of recursion compared to iterative approaches.**

   1. Higher Memory Usage
   Each recursive call is stored on the **call stack**, which consumes more memory than iteration, especially for deep recursions.

   ---

   2. Risk of Stack Overflow
   If recursion goes too deep (e.g., no base case or very large input), it can cause a **stack overflow** error due to excessive function calls.

   ---

   3. Slower Execution
   Recursive calls involve **function call overhead**, including saving stack frames and jumping between contexts, which can slow down execution compared to loops.

   ---

   4. Difficult to Debug and Understand
   Recursion can be harder to **trace**, especially for beginners, since the flow of control is not as linear and may require careful understanding of the call stack.

   ---

5. Not Always Necessary
Many problems that are solved using recursion can be solved more **efficiently** and **clearly** with an iterative approach, especially when tail recursion is not optimized by the compiler.

7. **Where can one encounter recursion in real-world applications outside of computer science?**

Recursion is not limited to programming—it appears in many natural and conceptual systems. Here are some key areas:

---

1. Mathematics
Recursive definitions are common in mathematics:
- **Factorials**, **Fibonacci sequence**, **fractals**, and **mathematical induction** are based on recursive logic.

---

2. Biology (Fractals in Nature)
Many biological structures show **self-similarity**:
- Examples: **Branches of trees**, **blood vessels**, **lungs**, **leaf patterns**, and **snowflakes** all exhibit recursive fractal patterns.

---

3. Language and Linguistics
Human languages often use **recursive grammar rules**:
- Example: "The cat **that chased the rat that stole the cheese** ran away." (Phrases nested within phrases)

---

4. Art and Architecture
Recursive patterns are seen in:
- **Mandalas**, **Islamic art**, **recursive tiling**, and **Escher's drawings**, where patterns repeat within themselves at different scales.

---

5. Economics and Finance
Some economic models use recursion to define future values:
- For example: **Compound interest**, **dynamic programming models**, and **decision trees**.

8. **Identify potential pitfalls to avoid when designing and implementing recursive functions**

When using recursion, it's important to write functions carefully to avoid common errors. Here are five key pitfalls:

---

1. Missing or Incorrect Base Case

Without a proper base case, the function will call itself **indefinitely**, leading to **stack overflow**.

Example mistake:

```
int fact(int n) {
   return n * fact(n - 1);  //  No base case!
}
```

---

2. Incorrect Recursive Step

If the recursive call does not move towards the base case, the function may **loop infinitely** or return incorrect results.

Example mistake:

Calling fact(n + 1) instead of fact(n - 1).

---

3. Redundant Computation

Some recursive functions (like naive Fibonacci) recompute the same values many times, which is **inefficient**.

Tip: Use **memorization** or **dynamic programming** when needed.

---

4. Stack Overflow for Large Inputs

Recursion consumes **stack space** for each call. Large inputs may cause the program to **crash** due to too many calls.

Example: factorial(100000) without tail recursion will likely overflow.

---

5. Overuse of Recursion Where Iteration is Better

Some problems are better solved with **loops**, as they are faster and use less memory.

Tip: Use recursion when it simplifies logic (e.g., tree traversal), not just because it's elegant.

9. **Show recursion using a c program to find the factorial of a number**

```
#include <stdio.h>

// Recursive function to calculate factorial
int factorial(int n) {
    if (n<=1)  // base case
        return 1;
    else
        return n * factorial(n - 1);  // recursive call
}

int main() {
    int num;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
```

```
    if (num < 0)
        printf("Factorial is not defined for negative numbers.\n");
    else {
        int result = factorial(num);
        printf("Factorial of %d is %d\n", num, result);
    }

    return 0;
}
```

10. Discuss the advantages and disadvantages of recursion with an example

Advantages of Recursion

1. **Simpler Code for Complex Problems:** Recursion provides a clean and elegant way to solve problems that have a **repetitive or hierarchical structure** (e.g., tree traversal, backtracking).
2. **Reduces Code Length:** Recursive functions are often **shorter and easier to understand** than iterative solutions, especially for problems like factorials, Fibonacci numbers, or tower of Hanoi.

Disadvantages of Recursion

1. **High Memory Usage:** Each recursive call uses **stack memory**, which can lead to **stack overflow** for large inputs.
2. **Slower Performance:** Recursive calls involve **overhead of function calls**, making them **slower** than loops in some cases (especially if no tail-call optimization is used).

Example: Factorial Using Recursion:

```
int factorial(int n) {
   if (n<=1)
      return 1;
   else
      return n * factorial(n - 1);
}
```

11. Explain the problem of recursion when the stack is full in detail

1. What is the Call Stack?

When a function (including a recursive one) is called, information like the return address, parameters, and local variables are stored in a region of memory called the **call stack**. Each function call adds a **stack frame** to the top of this stack.

---

2. How Recursion Uses the Stack

In recursion, every recursive call adds a new stack frame. For example, calling factorial(5) causes these nested calls:

factorial(5) → factorial(4) → factorial(3) → factorial(2) → factorial(1)

Each call waits for the next one to return, so stack frames pile up.

---

3. Stack Overflow Explained

The stack has a **limited size** (depends on system settings). If recursion is too deep or the base case is missing, the program keeps adding stack frames until the stack memory runs out. This causes a **stack overflow** error, which crashes the program.

---

4. Consequences of Stack Overflow

- **Program crash:** The system forcibly terminates the program.
- **Unpredictable behaviour:** May cause system instability.
- **Security risk:** Some exploits target stack overflow vulnerabilities.

---

5. How to Avoid Stack Overflow in Recursion

- Always define a **correct base case** to stop recursion.
- Use **tail recursion** where possible (some compilers optimize this).
- Limit the depth of recursion or switch to **iterative solutions**.
- Increase stack size (only if safe and necessary).

12. Differentiate between iterative approach and recursion.

Aspect	Iterative Approach	Recursive Approach
Definition	Repeats a block of code using loops (for, while)	Function calls itself to solve smaller instances of the problem
Control Structure	Uses loops to repeat operations	Uses function calls and base cases
Memory Usage	Uses constant memory for loop control variables	Uses additional stack memory for each recursive call
Performance	Generally faster due to less overhead	May be slower due to function call overhead
Termination	Loop terminates based on loop condition	Recursion ends when base case is met
Code Complexity	Can be more complex for problems like tree traversal	Often simpler and more intuitive for problems with repetitive sub-structure
Risk of Errors	Less risk of stack overflow	Risk of stack overflow if recursion is too deep or base case is missing

13. Associate recursion with top-down approach with a suitable diagrammatic representation

1. Top-Down Approach Explanation

- The **top-down approach** is a problem-solving strategy that starts with the **overall problem** and breaks it down into smaller, more manageable **sub-problems**.
- This continues recursively until the sub-problems become simple enough to solve directly (base case).
- **Recursion** naturally fits this approach since a recursive function solves a problem by calling itself on smaller inputs.

---

2. How Recursion Fits Top-Down Approach

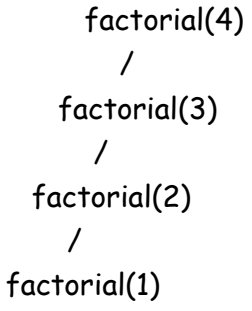
- In recursion, you begin solving the **big problem**.
- Then, break it into **smaller instances** of the same problem.
- Continue breaking down recursively.
- Combine solutions of smaller sub-problems to get the final answer.

Example: Factorial (Top-Down Recursion)

- To calculate factorial(4), break it down as:

factorial(4)
= 4 * factorial(3)
= 4 * (3 * factorial(2))
= 4 * (3 * (2 * factorial(1)))
= 4 * 3 * 2 * 1
= 24

Diagrammatic Representation:



14. Illustrate recursion on finding the sum of nth even number with a suitable flowchart and an algorithm.

Algorithm:

Algorithm SumEven(n)

Input: n (number of even numbers)
Output: sum of first n even numbers

```
Begin
   if n == 1 then
      return 2
   else
      return 2*n + SumEven(n - 1)
End
```

15.